

Simple iteration method to approximate **v** vector in **Av=u** equations.

This solution proves that values of **v -> J_(1, matrixsize)**, having **A = J_matrixsize + (2 * Id)** and **u = J_(1, matrixsize) * (matrixsize + 1)**

Build

```
make
```

Run

```
# Enter the number of threads and the size of quadratic matrix.
./dgemv num_threads matrix_size
```

Use `make clean` to remove the executable file.

Report

Node description

Model name:	Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
CPU(s) :	80
Server name:	ProLiant XL270d Gen10
NUMA node(s) :	2
NUMA node0 CPU(s) :	0-19,40-59
NUMA node1 CPU(s) :	20-39,60-79
node 0 size:	385636 MB
node 0 free:	105947 MB
node 1 size:	387008 MB
node 1 free:	232013 MB
OS:	Ubuntu 22.04.5 LTS

Scaling

Selected schedule	Matrix size (M=N)	Thread amount										
		1	4		8		16		20		40	
		T(1), s	T(4), s	S(4)	T(8), s	S(8)	T(16), s	S(16)	T(20), s	S(20)	T(40), s	S(40)
Static	400	~0.11	~0.042	~2.68	~0.06	~1.81	~0.27	~0.85	~0.12	~0.98	~0.1	~1.10
	1000	~0.33	~0.104	~3.32	~0.07	~6.31	~0.16	~4.48	~0.2	~2.55	~0.18	~1.83
	2000	~1.13	~0.330	~4.26	~0.3	~6.88	~0.09	~13.56	~0.08	~12.79	~0.14	~7.67
	4000	~4.37	~1.22	~3.95	~1.17	~5.16	~0.38	~12.58	~0.31	~14.01	~0.22	~20.23
Guided	400	N/A	~0.047	~2.41	~0.07	~1.46	~0.25	~0.89	~0.16	~0.72	~0.16	~0.68
	1000		~0.097	~3.57	~0.08	~5.36	~0.08	~8.50	~0.09	~5.33	~0.19	~1.76
	2000		~0.336	~4.18	~0.2	~10.55	~0.11	~11.49	~0.10	~10.29	~0.14	~7.79
	4000		~1.2	~4.05	~1.12	~5.38	~0.60	~7.90	~0.33	13.06	~0.30	~15.07

Conclusion on thread usage

All the paralleling rules were linked to a specific loops of iteration process:

```
#pragma omp parallel for num_threads(N) schedule(sch)
```

For small matrices, schedule changes (and the paralleling rules at all) couldn't make any impact on processing.

Looking at the 1000x1000 matrix size, there's definitely a positive effect on guided thread scheduling. However, as the matrix was expanding, guided scheduling went from approx. equally effective as static scheduling (2000x2000) to being more of a slowing solution.

Furthermore, by increasing the thread amount, both scheduling rules tend to have the same performance in speedup.

